

Music-Gen — Rules Schema v1 (clone-1 merge)

2026-08-28

Contents

Music-Gen — Rules Schema v1 (clone-1 merge)	1
Abstract	1
1. Goal and scope	1
2. The common envelope	2
3. The five typed parameter blocks	2
4. rule_id — content addressing	3
5. Supersede semantics — append-only, never in-place	3
6. What the validator rejects	3
7. Consumer interface	4
8. Test suite	4
9. What this branch does not attempt	5
10. Open items handed to the root conductor	5
Appendix: Implementation Details	5

Music-Gen — Rules Schema v1 (clone-1 merge)

Abstract

This branch designs and lands the version-1 rule schema that will carry every extracted musical pattern from score analysis into the generation stage of the Music-Gen pipeline. The schema covers five rule categories — harmonic, rhythmic, melodic, form, and arrangement — and fixes the contract that every rule instance, whatever its category, must expose: a stable identifier, a typed parameter block, an explicit time-and-place scope, pointers back to the transcription events it was derived from, and a numeric confidence in the closed interval [0, 1]. A companion Python validator enforces the contract, a small append-only ledger records rule instances and their supersessions, and a synthetic-instance suite plus a planted-invalid suite together demonstrate that the contract behaves as specified.

The deliverable stands ready to consume real rules the moment a merged full-song score becomes available upstream. No part of it depends on rated audio, so the branch was not blocked by the workspace’s current egress restriction on YouTube downloads.

1. Goal and scope

The upstream campaign plan divides the rule-mining stage into two halves: a schema half (define the on-disk shape and the invariants) and an extraction half (populate it from real scores). This branch owns the schema half only. The extraction half is deliberately out of scope: it cannot begin until an earlier milestone produces a merged full-song score on at least one seed piece, and that milestone itself waits on transcription. The consumer interface documented here is the seam at which extraction will plug in.

The five rule categories were fixed by the campaign prompt. The concrete design choices —

what fields are common to all rules, what parameters each category carries, how identifiers are generated, and how a wrong rule is retracted — are the substantive content of this branch.

2. The common envelope

Every rule instance, regardless of category, carries the same envelope of fields:

- **rule_id** — a content-addressed 16-hex-character identifier (see §4). Rules with identical content collapse to the same id; rules that differ in any observable field get different ids.
- **rule_type** — one of the five closed category names: harmonic, rhythmic, melodic, form, arrangement. Any other value is rejected at validation time (see §6).
- **schema_v** — the schema major version. Fixed at 1 for this release. A future breaking change increments this and forks the schema file.
- **extractor_version** — a free-form string identifying the extractor build that produced the rule. Recorded, never interpreted, so that a downstream reader can trace a rule back to the code that made it.
- **scope** — one of three shapes: song (whole piece), section (named span with start/end seconds), or measure (measure-range with start/end measure numbers and start/end seconds). The two temporal coordinates are both required and both bounded — measure numbers are integers ≥ 1 with $\text{end} \geq \text{start}$; seconds are non-negative reals with $\text{end} \geq \text{start}$.
- **provenance_pointers** — a non-empty list of pointers into the transcription output. Each pointer names one or more transcription event ids (the atomic notes and downbeats produced by the transcriber) and, optionally, a measure range. A rule with no provenance is rejected: the audit-trail invariant is structural, not advisory.
- **confidence** — a real number in the closed interval $[0, 1]$. Values outside this range are rejected.
- **parameters** — a typed block whose shape depends on **rule_type**; see §3.
- **superseded_by** — optional pointer to a successor **rule_id**. Absent for live rules; set exactly once when a rule is retracted (see §5).

3. The five typed parameter blocks

Each **rule_type** corresponds to a distinct sub-schema for parameters. The categories were chosen to partition the space of musical patterns the generation stage will eventually need to sample from:

Harmonic rules carry a chord-quality vocabulary, a root sequence, and an optional key/mode. Example instances in the suite include a ii-V-I cadence pattern, a modal borrow from parallel minor, and a functional prolongation across a section.

Rhythmic rules carry a meter, a beat subdivision, and either an accent pattern (positional) or a groove template (a probability vector over sixteenth-note positions). Example instances include a straight-eighth backbeat, a 3-3-2 clave, and a swung-eighth pattern with a specified swing ratio.

Melodic rules carry an interval-class contour, a range in semitones, and an optional pivot-note anchor. Example instances include a stepwise-descent contour bounded by a fifth and a leaping arpeggiation of the underlying chord.

Form rules carry a labeled-section sequence (e.g. A B A B C B), section-length statistics, and repetition/variation flags between labeled sections. Example instances cover verse-chorus alternation, a modified-repeat outro, and a through-composed span.

Arrangement rules carry a per-section instrument-family activity map (each instrument

marked active / sparse / silent) and, optionally, a density envelope. Example instances include a “drop the drums for the second verse” pattern and a “strings enter on the bridge” pattern.

Each parameter block is validated by its own JSON-schema sub-object. The validator dispatches on `rule_type` and applies the matching sub-object; a mismatch between the declared `rule_type` and the parameter-block shape is a validation error.

4. `rule_id` — content addressing

Rule ids are derived from a canonical serialization of the rule’s observable content: `rule_type`, `schema_v`, `scope`, `provenance_pointers`, `parameters`. The `extractor_version`, `confidence`, and `superseded_by` fields are deliberately excluded from the hash — they are metadata about the observation, not part of the observation.

The canonical serialization sorts object keys, normalizes numeric representations, and encodes as UTF-8 JSON with no whitespace. The id is the first 16 hex characters of the SHA-256 digest of that byte sequence, prefixed with `rule_`. Two extractors that observe the same pattern at the same scope with the same provenance will therefore emit the same `rule_id`; a difference in any of those fields produces a different id.

Determinism was checked directly: the twenty-five synthetic instances were written to disk, re-read, and re-hashed; every id reproduced exactly. This is the round-trip determinism invariant called for by the branch objective.

5. Supersede semantics — append-only, never in-place

Rules are corrected by appending, never by editing. When an extractor decides that an earlier rule was wrong (say, because a later pass produced a better analysis of the same span), it appends a new rule with a new `rule_id` and then writes a supersede entry to the ledger. The old rule’s `superseded_by` field points to the new rule’s id; the old rule’s content is otherwise untouched.

The rationale is auditability: any downstream consumer can reconstruct the full history of an analysis by replaying the ledger in append order. Chained supersessions are permitted — rule A → rule B → rule C — and the ledger walk resolves them transitively. Cycles are rejected at ledger-append time.

The ledger itself is a JSON-Lines file (`data/rules/ledger.jsonl`). Each line records either a rule-emit event or a supersede event, with wall-clock timestamp and the emitting extractor’s version. Append-only is enforced by convention (single-writer during the campaign loop); a future change to concurrent extractors would need file locking, and that is noted for the merge report but not implemented here.

6. What the validator rejects

The validator was written as a two-layer object: a JSON-Schema (2020-12) layer that catches structural errors, and a Python layer that catches invariants JSON Schema cannot express (id determinism, supersede-cycle detection, and cross-field consistency between `rule_type` and `parameters` shape).

Four classes of planted-invalid instances were checked, all rejected as expected:

- **Unknown `rule_type`** — a rule with `rule_type`: “dynamics” (not one of the five). Rejected at the enum check. The policy is reject, not ignore: an unknown type is treated as a

possible schema-version drift and must be resolved by the emitter, not silently dropped by the reader.

- **Missing provenance** — a rule with an empty `provenance_pointers` list. Rejected at the min-length check. There is no such thing as a valid rule without a pointer back to the observation that produced it.
- **Out-of-range confidence** — instances with `confidence: 1.5` and `confidence: -0.1`. Both rejected at the numeric-range check.
- **Duplicate rule_id** — a second rule appended to the ledger with the same id as an existing live rule. Rejected at ledger-append time. (A duplicate that resolves to the same content is silently deduplicated rather than rejected; the failure case above uses a duplicate id whose content has been tampered with, which is a stronger signal of corruption.)

Additional invariants checked by the Python layer: scope bounds (`end ≥ start` for both measures and seconds), non-negativity of second-valued fields, and the rule/parameter type match.

7. Consumer interface

The seam at which the extraction half of this milestone (and, later, the generation stage) will plug in is small and explicit:

- **Read path:** load the schema (`scripts/rules/schema/rules_v1.json`), load the ledger (`data/rules/ledger.jsonl`), and walk it to produce the current live rule set (rules with no `superseded_by` still active at ledger head).
- **Write path:** construct a rule object matching the envelope in §2 and the parameter block for its `rule_type` (§3); compute its `rule_id` with the helper in `scripts/rules/rule_id.py`; validate with `scripts/rules/validate.py`; append via `scripts/rules/ledger.py`.
- **Supersede path:** as §5. The helper emits both the new rule and the supersede pointer in a single ledger transaction so partial writes cannot leave the ledger inconsistent.

The extraction half of the milestone, once unblocked upstream, will import these three helpers unchanged. Nothing about the extractor’s internal design is exposed on this seam.

8. Test suite

Twenty-five synthetic instances were authored — five per rule type — and stored under `scripts/rules/schema/examples/<type>/.` Each instance is a small, hand-audited example intended to exercise a distinct sub-pattern within its category. The suite is not a corpus for training; it is a set of fixtures for the validator and for downstream consumers to test against.

The `tests/test_rules_schema.py` file (413 lines) runs the following checks on every push:

1. All twenty-five synthetic instances validate.
2. Each instance round-trips: write → read → identical bytes.
3. Each of the four planted-invalid classes above is rejected with a specific, non-empty error message.
4. `rule_id` determinism reproduces from scratch on every run.
5. Supersede transitivity ($A \rightarrow B \rightarrow C$) resolves correctly on a fresh temp ledger.
6. A duplicate-id append is rejected at ledger level.

The audit trail records 25/25 synthetic instances passing and 11 planted-invalid checks rejected, with the additional five planted invalids exercising edge cases within the four documented classes (e.g. a negative measure number, a section scope with `end_sec < start_sec`).

At branch merge, the cross-branch integration suite grew from 68 to 90 checks with the rules-

schema material folded in; all 90 pass.

9. What this branch does not attempt

- **Extraction from real scores.** Blocked upstream by transcription and full-score merge. When those land, the seam in §7 is the entry point.
- **Concurrent writers on the ledger.** The single-writer assumption is enforced by convention only. If the campaign later runs multiple extractors in parallel, file locking is the small change required — noted, not implemented.
- **A binary or columnar encoding.** JSON-Lines is chosen for auditability and diff-friendliness. If read throughput ever becomes a bottleneck, a derived index can be built without changing the on-disk contract.
- **Rated-audio-dependent work.** The rated-playlists corpus is registered with full provenance but audio downloads remain blocked by workspace egress policy. Nothing in this branch touches audio, so nothing here is affected. The harvest script should continue to be retried in case egress opens.

10. Open items handed to the root conductor

The following items are surfaced to the parent so they can be picked up at the next appropriate stage:

- **Schedule the extraction half** once the full-score-merge milestone completes. Its consumer interface is §7 above.
- **Two pre-existing errors in the plan-consistency checker** (at the two ledger lines that name this milestone and the ear-training preparation milestone) are a parser-side mismatch in the checker itself — both plan entries are present at the expected locations. This belongs to a future infrastructure-owner cycle; it is not urgent and did not affect this branch.
- **Orphan-artifact warnings** on the new `scripts/rules/**` and companion test file are the expected artifact of running inside a fan-out clone; they resolve automatically when the shadow ledger is folded back into the main ledger at merge.
- **Reusable pattern.** The shape that landed here — a schema file, a two-layer validator, a synthetic-instance matrix, a planted-invalid matrix, and a small documented consumer seam — is a clean template for any future schema-first branch in this campaign. Noted for reuse; not itself a task.

Appendix: Implementation Details

Files created or modified during this branch (paths relative to workspace root):

- `scripts/rules/schema/rules_v1.json` (222 lines) — JSON Schema 2020-12 source of truth.
- `scripts/rules/schema/rules_v1.yaml` (370 lines) — YAML equivalent, generated from the JSON by `scripts/rules/schema/build_yaml.py` and checked in for readability.
- `scripts/rules/schema/README.md` — reader-facing overview of the schema layout.
- `scripts/rules/schema/examples/<type>/*.json` — 5 instances per category × 5 categories = 25 files.
- `scripts/rules/schema/build_examples.py` — regenerates the example set deterministically.
- `scripts/rules/rule_id.py` (65 lines) — canonical serialization and id derivation.
- `scripts/rules/validate.py` (186 lines) — two-layer validator.
- `scripts/rules/ledger.py` (193 lines) — append-only ledger with supersede support and cycle detection.
- `scripts/rules/__init__.py` (5 lines) — package marker.

- `data/rules/ledger.jsonl` — the ledger file itself; initialized with the synthetic instances so downstream consumers have something to read against.
- `tests/test_rules_schema.py` (413 lines) — full test suite.
- `docs/rules_schema_report.md` (293 lines) — companion document at the milestone level; this report is the cycle-level companion.

Cross-branch integration. Independently reproduced in the branch’s earlier cycles: 25/25 synthetic instances passing, 11 planted-invalid rejections, `rule_id` determinism reproduced from scratch on a fresh interpreter, supersede transitivity confirmed on a fresh temp ledger, cross-branch integration suite green at 90/90.

Cycle-level sessions. Cycles 1-2 of this clone: researcher `bfe4b5ec-f7fa-49ef-bd2e-6157a3912467` and `84e4881e-1eb0-4304-8e19-fe3aa409c137`; worker `3f609512-5fa6-415f-93dc-15c3c863f612` and `d646a9ad-ca9a-4a50-aaaa-596efddf4b50`; auditor `e8792cad-c92b-41ec-97d0-2ddfe918173a` and `40f669e0-4e4e-4fb7-9cab-aaa4f18c4827`. The branch as a whole ran seven cycles; the substantive convergence was reached earlier and the terminal cycle emitted a single scope-closure event with an immediately-archived one-shot helper, introducing no new artifacts.

Ledger events at branch close. One authorized rollup event (`_run/clone-1-scope-complete`) plus the pre-existing milestone-level validation event from earlier in the branch. No other events emitted.

Merge verdict. The branch objective — schema, validator, synthetic-instance suite, planted-invalid suite, round-trip determinism, supersede pattern, consumer interface — was met and independently confirmed. The extraction half is out of scope and correctly deferred.

validated